

RESEARCH ARTICLE | NOVEMBER 20 2014

Nonorthogonal orbital based N -body reduced density matrices and their applications to valence bond theory. IV. The automatic implementation of the Hessian based VBSCF method

Xun Chen; Zhenhua Chen; Wei Wu



J. Chem. Phys. 141, 194113 (2014)

<https://doi.org/10.1063/1.4901729>



Articles You May Be Interested In

Nonorthogonal orbital based N -body reduced density matrices and their applications to valence bond theory. II. An efficient algorithm for matrix elements and analytical energy gradients in VBSCF method

J. Chem. Phys. (April 2013)

A quadratically convergent VBSCF method

J. Chem. Phys. (February 2013)

λ -DFVB(U): A hybrid density functional valence bond method based on unpaired electron density

J. Chem. Phys. (May 2022)

04 March 2025 08:52:54



Nanotechnology & Materials Science



Optics & Photonics



Impedance Analysis



Scanning Probe Microscopy



Sensors



Failure Analysis & Semiconductors



Unlock the Full Spectrum.
From DC to 8.5 GHz.

Your Application. Measured.

[Find out more](#)



Nonorthogonal orbital based N -body reduced density matrices and their applications to valence bond theory. IV. The automatic implementation of the Hessian based VBSCF method

Xun Chen, Zhenhua Chen, and Wei Wu^{a)}

The State Key Laboratory of Physical Chemistry of Solid Surfaces, Collaborative Innovation Center of Chemistry for Energy Materials, Fujian Provincial Key Laboratory of Theoretical and Computational Chemistry, and College of Chemistry and Chemical Engineering, Xiamen University, Xiamen, Fujian 361005, China

(Received 7 July 2014; accepted 3 November 2014; published online 20 November 2014)

In this paper, the Hessian matrix of valence bond (VB) self-consistent field (VBSCF) energy with respect to orbitals are evaluated by applying the nonorthogonal orbital based N -body reduced density matrices, which was presented in Paper I. To this end, an automatic formula/code generator (AFCG) is developed; with which the matrix elements between internally contracted excited configurations of VB wave function and the corresponding codes are generated automatically. Compared to the tedious manual formula deducing and implementing, AFCG is much more convenient and efficient, and enables us to avoid troublesome debugging. With the help of AFCG, the Hessian-based Newton-Raphson algorithm is implemented for the VBSCF orbital optimization. Test calculations indicate that the Newton-Raphson algorithm converges quadratically and has much better convergence behavior than the gradient-based LBFGS algorithms. Furthermore, a combined approach with LBFGS and Newton-Raphson algorithms is applied to reduce the total CPU time of the calculation. © 2014 AIP Publishing LLC. [<http://dx.doi.org/10.1063/1.4901729>]

I. INTRODUCTION

In Paper I, we proposed the reduced density matrix (RDM) approach for the nonorthogonal orbital based many-electron theory.¹ The approach has been applied to the nonorthogonal valence bond (VB) theory in Paper II for the energy gradient based VB self-consistent field (VBSCF) method.² Furthermore, in Paper III, the approach was extended to the valence bond second-order perturbation theory (VBPT2) by means of internal contraction technique, and thus the method is named as icVBPT2.³ The three papers are called Papers I, II, and III, respectively, hereafter. In this paper, we further apply the RDM-based approach to the evaluation of the analytical Hessian of VBSCF energy.

In modern VB theory, a classical question is how to optimize the VB wave function. In the VBSCF method,⁴ the linear coefficients of VB structures and the VB orbitals are optimized simultaneously. The former can be optimized routinely by solving a secular equation. However, the optimization of VB orbitals is much more difficult than the structural coefficients. This is because the nonorthogonality of the VB orbitals brings an inevitable difficulty in the optimization procedures, compared to the analogous molecular orbital (MO) based method, in which orthogonality condition is imposed on MOs. A practical approach for the orbital optimization is to find such a stationary point on the energy surface by direct searching strategies, which uses the analytical energy gradients only. To do so, we have deduced the formulas using the transition Fock matrix based approach.⁵ In this approach, the

time-consuming integral transformation is avoided; alternatively the analytical energy gradients were obtained from the transition Fock matrices of determinant pairs. However, the computational cost of each transition Fock matrix is the same as the construction of Fock matrix and thus it is less efficient when the number of determinant pairs involved in the calculation is large. In the RDM-approach presented in Paper II,² we express the energy gradients in terms of the tensor contractions of RDMs and electron integrals and apply to the LBFGS algorithm,⁶ avoiding the direct evaluation of large numbers of Hamiltonian matrix elements between excited VB determinants. As a consequence, the RDM-based LBFGS algorithm shares a lower scaling compared to our previous algorithms. However, the common disadvantage of the LBFGS-based optimization methods is that the convergence behaviors are usually not satisfactory, due to lack of a proper approximate Hessian. Especially, for some larger systems, it may take hundreds of iterations to achieve convergence.

In modern quantum chemistry, a common strategy to address the convergence problem is to use either the exact or approximate Hessian-based optimization methods. In MO-based MCSCF methods, this has been extensively investigated for many years,^{7–21} and some efficient algorithms have been proposed, such as the trust region Newton-Raphson method,¹³ the norm extended optimization scheme,¹⁸ and the augmented Hessian method.^{19–21} In VB methods, numerous efforts have been paid for the orbital optimization of spin-coupled valence bond (SCVB) method.^{22–26} Gerratt and co-workers^{27,28} introduced the level-shifted Hessian method to optimize the orbital and spin-coupling coefficients simultaneously. This second-order optimization scheme is further

^{a)} Author to whom correspondence should be addressed. Electronic mail: weiwu@xmu.edu.cn. Fax: (86)592-2183047.

improved by using trust region scheme and incorporating the core-valence separation.²⁹ Cooper and co-workers³⁰ developed a Hessian-based optimization scheme for the SCVB wave function by transforming it to the complete active space SCF (CASSCF) one. Lately, the CASVB strategy, the level-shift and trust region schemes were applied for the optimization of SCVB wave function for excited states.³¹ They also used the augmented Hessian based scheme for the optimization of the spin-coupled wave function constructed with biorthogonal orbitals.³² Recently, the augmented Hessian based scheme was developed by Rashid and van Lenthe³³ for the optimization of VBSCF wave function.

The purpose of this paper is to accelerate the convergence of VBSCF orbital optimization procedure. To do so, the RDM-based approach is applied for the evaluation of Hessian and thus the Newton-Raphson algorithm can be implemented for orbital optimization. However, as is well known, the formula deduction and code implementation for the Hessian matrix are very tedious and complicated due to the use of nonorthogonal orbitals in VB methods. To overcome this difficulty, the automatic formula/code generation (AFCG) technique is also developed for nonorthogonal orbitals in this paper. The AFCG technique has been used in some quantum chemistry programs to implement sophisticated formulas into codes since 1990. Janssen and Schaefer,³⁴ Li and Paldus,³⁵ and Nooijen and Marcel³⁶ applied the automatic code generation (ACG) technique to the implementation of the coupled cluster methods. After that, the tensor contraction engine (TCE), which is a powerful tool to automatically deduce the equations of the second-quantized many-electron theory and further to synthesis efficient parallel computer programs on the basis of these equations, is developed under the collaboration of several groups.³⁷ After the first prototype of TCE emerged in 2003,³⁸ great efforts have been made by the TCE developers to improve the performance of code generated by the TCE.³⁹ The TCE has been used to implement the coupled cluster methods in the NWChem package.⁴⁰ Besides, the LIBINT electronic integral calculation library, developed by Fermann and Valeev, uses the automatic code generation technique to implement efficient electronic integral calculation routines.⁴¹

However, all these existing AFCGs are based on the fact that MOs are orthogonal, which is not true for VB theory. As a consequence, the existing AFCGs could not be applied to *ab initio* VB methods. Thus, to implement the Hessian based algorithm for VBSCF, it is essential to develop an AFCG tool for the nonorthogonal orbital based many-electron theory.

The paper is organized as follows: In Sec. II, the VB-SCF method is briefly reviewed, then the VBSCF energy gradient and Hessian, and Newton-Raphson optimization algorithm are introduced. After that, in Sec. III, the main features of the automatic formula/code generator (AFCG), the implementation of necessary matrix elements in the VBSCF orbital Hessian with AFCG, and the program procedure of the Newton-Raphson algorithm for VB orbital optimization, are presented. Then, some test calculations are carried out to prove the feasibility of the AFCG and to test the performance of the Newton-Raphson algorithm. Finally, the implementation details of the AFCG are discussed in the Appendix.

II. THEORY AND METHODOLOGY

A. The VBSCF method

In the VBSCF method, the wave function is expressed as a linear combination of VB structures,

$$\Psi = \sum_K C_K \Phi_K, \quad (1)$$

where Φ_K and C_K are VB structure and its corresponding coefficient. The VB structure, Φ_K , may be written in terms of the Slater determinants build upon VB orbitals $\{\phi_p\}$, which are expressed as a linear combination of basis functions $\{\chi_\mu\}$,

$$\phi_p = \sum_\mu T_{\bullet p}^\mu \chi_\mu. \quad (2)$$

As the same in the previous papers of the series,² indices i, j, k, l denote the doubly occupied inactive orbitals and indices t, u, v, w, x, y denote the partially occupied active orbitals, while the unoccupied virtual orbitals have indices a, b, c, d , and indices p, q, r, s represent any orbitals.

The VBSCF energy (E) and the coefficients of VB structures are determined by solving the secular equation,

$$\mathbf{H}\mathbf{C} = E\mathbf{M}\mathbf{C}, \quad (3)$$

where $\mathbf{H}$ and $\mathbf{M}$ are the Hamiltonian and overlap matrices of VB structures, respectively, and $\mathbf{C}$ is the column matrix containing $\{C_K\}$.

B. The VBSCF energy gradient and Hessian

The gradient of VBSCF energy with respect to VB orbital replacement, $R_{\bullet q}^p$,

$$\phi_q^{\text{new}} = \sum_p R_{\bullet q}^p \phi_p^{\text{old}}, \quad (4)$$

can be obtained by using internal contraction technique,^{1,2}

$$\xi_p^{\bullet q} = \frac{\partial E}{\partial R_{\bullet q}^p} = 2 \langle \Psi | (H - E) | \Psi_p^{\bullet q} \rangle, \quad (5)$$

where H is the Hamiltonian operator and E is the total energy, $\Psi_p^{\bullet q}$ represents the internally contracted singly excited wave function (icSEWF), which is generated by applying the one-body reduced density operator (RDO, $E_p^{\bullet q}$) onto the VBSCF wave function,

$$\Psi_p^{\bullet q} = E_p^{\bullet q} \Psi. \quad (6)$$

Furthermore, the orbital Hessian can be written as⁴²

$$\begin{aligned} \Xi_{qs}^{\bullet\bullet pr} &\equiv \frac{\partial^2 E}{\partial R_{\bullet p}^q \partial R_{\bullet r}^s} = 2 \langle (\Psi_q^{\bullet p} | H - E | \Psi_s^{\bullet r}) \\ &+ \langle \Psi | H - E | \Psi_{qs}^{\bullet\bullet pr} \rangle \\ &- 4 \langle (\Psi | H - E | \Psi_s^{\bullet r}) \langle \Psi | \Psi_q^{\bullet p} \rangle \\ &+ \langle \Psi | H - E | \Psi_q^{\bullet p} \rangle \langle \Psi | \Psi_s^{\bullet r} \rangle \rangle. \end{aligned} \quad (7)$$

In Paper II, the RDM approach was applied to the evaluation of the gradients,² and in this paper it is further applied to the evaluation of Hessian matrix. It can be seen from Eq. (7) that

Hessian matrix elements include the following six types:

$$\langle \Psi | \Psi_q^{\bullet p} \rangle, \quad (8)$$

$$\langle \Psi | \Psi_{qs}^{\bullet\bullet pr} \rangle, \quad (9)$$

$$\langle \Psi_q^{\bullet p} | \Psi_s^{\bullet r} \rangle, \quad (10)$$

$$\langle \Psi | H | \Psi_q^{\bullet p} \rangle, \quad (11)$$

$$\langle \Psi | H | \Psi_{qs}^{\bullet\bullet pr} \rangle, \quad (12)$$

$$\langle \Psi_q^{\bullet p} | H | \Psi_s^{\bullet r} \rangle. \quad (13)$$

The matrix elements between the VBSCF wave function and icSEWF, Eqs. (8) and (11), are also contained in Eq. (5), which were implemented in the Xiamen Valence Bond (XMVB) package.⁴³ Thus, in this paper, only the matrix elements, Eqs. (9), (10), (12) and (13), need to be considered, where the internally contracted double excited wave functions (icDEWFs) are involved in Eqs. (9) and (12). Traditionally, the evaluations of Eqs. (8)–(13) require expanding the VBSCF wave function, icSEWFs and icDEWFs, in terms of VB structures (or nonorthogonal orbital-based determinants), and then summing up the matrix elements between them. However, with the enhanced Wick's theorem, shown in Paper I,¹ these matrix elements could be expressed in terms of tensor contractions of n -RDMs and electronic integrals, avoiding the expensive computation of matrix elements of VB structures. As a result, the cost of the calculation of VBSCF orbital Hessian might reduce greatly, because the n -RDMs can be evaluated efficiently by using the α - and β -string-based algorithm given in Paper II.²

Considering the facts that there are three classes of orbitals, inactive, active, and virtual ones, and there are four indices in each matrix element, we shall have totally $3^4 = 81$ situations to deal with for one type of matrix element. Obviously, it would be very tedious to deduce the expressions explicitly for all matrix elements and to implement them into Fortran codes by hands. Alternatively, a practical and efficient way is to develop an AFCG tool, by which all formulations required for the Hessian are accomplished and then implemented automatically.

C. The Newton-Raphson algorithm

For given structure coefficients, the Taylor expansion of VBSCF energy up to second order is expressed as

$$E(\phi + \mathbf{x}) = E(\phi) + \xi^+ \mathbf{x} + \frac{1}{2} \mathbf{x}^+ \Xi \mathbf{x}, \quad (14)$$

where ξ and Ξ are the energy gradients and Hessian arranged in a column vector and a matrix, respectively, and $\mathbf{x}$ is the orbital increment vector,

$$\phi^{\text{new}} = \phi^{\text{old}} + \mathbf{x}. \quad (15)$$

In Eq. (15), ϕ^{old} and ϕ^{new} represent the orbital coefficient vectors before and after a Newton-Raphson step, respectively. In the Newton-Raphson algorithm, the stationary point of the VBSCF energy can be determined by constructing a sequence of approximated quadratic models, Eq. (14). By setting the differential of Eq. (14) to zero, one has the Newton-Raphson equation,⁴⁴

$$\Xi \mathbf{x} + \xi = 0. \quad (16)$$

Equation (14) has a true minimum only if the Hessian matrix is positive definite. However, this is not the case for VBSCF energy Hessian with respect to the orbital coefficients. The redundant variables in a general VBSCF wave function at least include following orbital excitations: (1) from occupied to inactive; (2) from an active orbital to itself; and (3) from occupied orbital of one irreducible representation to orbital with different irreducible representation of the point group. Other redundant variables corresponding to excitations within the active orbital subspace are more difficult to be determined, since they are coupled with the VB structure coefficients and also depend on the specific types of the used orbitals: hybrid atomic orbitals (HAOs), bond-distorted orbitals (BDOs), and overlap enhanced orbitals (OEOs).

It is well known that the Hessian matrix is semi-positive-definite at the stationary point. In other words, the eigenvalues corresponding to the redundant variables vanish at the stationary point. These eigenvalues might be negative when the VBSCF wave function is far from converged. Fortunately, such redundant variables can be determined directly by diagonalizing the Hessian matrix,

$$\mathbf{U} \Lambda \mathbf{U} = \Xi, \quad (17)$$

and removing the negative and small eigenvalues.⁴⁵ Moreover, this diagonalization method also removes other negative eigenvalues due to the pronounced error of the approximation quadratic models when the wave function is far from converged. As shall be seen in the test calculations, the above scheme leads to a quadratically convergent VBSCF method.

Substitute Eq. (17) into Eq. (16), one has

$$\Lambda \tilde{\mathbf{x}} + \tilde{\xi} = 0, \quad (18)$$

where

$$\tilde{\mathbf{x}} = \mathbf{U}^+ \mathbf{x} \quad (19)$$

and

$$\tilde{\xi} = \mathbf{U}^+ \xi. \quad (20)$$

The predicted energy change by the Newton-Raphson algorithm can be expressed as

$$\Delta E = -\frac{1}{2} \tilde{\xi}^+ \tilde{\mathbf{x}}. \quad (21)$$

III. IMPLEMENTATION

A. The automatic formula/code generator

As mentioned above, to implement the Hessian based algorithm for the VBSCF method, it is indispensable to develop

an AFCG tool for the nonorthogonal VB theory. The AFCG was written with the Python programming language.⁴⁶

The AFCG is composed of two modules, AFG (automatic formulas generator) and ACG (automatic code generator). AFG reads the Hamiltonian/Overlap matrix elements required to calculate as input, parses the input, and then accomplishes the formula deduction to get the resulting formulas by utilizing the enhanced Wick's theorem. The output formulas can be printed either in the plain text form or in LaTeX input file format, by which a PDF file can be easily produced.

After generating the formulas, the ACG module will generate corresponding codes automatically. The output codes are in standard Fortran form and are compiled into the XMVB package⁴³ with a general Fortran compiler such as GFortran or IFORT. The implementation details of AFCG are discussed in the Appendix.

B. The matrix elements deduction and implementation

As mentioned in the Sec. II B, there are totally four kinds of matrix elements, which include Hamiltonian and overlap matrix elements between the VBSCF wave function and icDEWF, and matrix elements between two icSEWFs. Each kind of matrix element has four indices and each index could have three types, inactive, active, and virtual ones. As thus, there are totally $4 \times 3^4 = 324$ matrix elements that are required for the evaluation of Hessian. With the help of the AFCG, all these matrix elements are deduced and implemented into Fortran codes. Eqs. (22)–(25) show the resulting formulas for the overlap and Hamiltonian containing four active indices, while all the remaining non-zero formulas are given in the supplementary material.⁴⁷

$$\langle \Psi | \Psi_{TU}^{\bullet\bullet VW} \rangle = D^{VW,ut} s_{Ut} s_{Tu}, \quad (22)$$

$$\langle \Psi_T^{\bullet U} | \Psi_V^{\bullet W} \rangle = D^{UW} s_{TV} + D^{Ut,uW} s_{Vt} s_{Tu}, \quad (23)$$

$$\begin{aligned} \langle \Psi | H | \Psi_{TU}^{\bullet\bullet VW} \rangle = & h_{Uu} D^{VW,tu} s_{Tt} + h_{Tu} D^{VW,ut} s_{Ut} + h_i^{\bullet i} D^{VW,tu} s_{Uu} s_{Tt} + h_{vw} D^{VWv,utw} s_{Ut} s_{Tu} \\ & + g_{TUtu} D^{VW,tu} - g_{Ui\bullet u}^{\bullet\bullet i} D^{VW,tu} s_{Tt} + 2g_{Uiu}^{\bullet\bullet\bullet i} D^{VW,tu} s_{Tt} + g_{Uuvw} D^{VWu,tvw} s_{Tt} \\ & - g_{Ti\bullet u}^{\bullet\bullet i} D^{VW,ut} s_{Ut} + 2g_{Tiu}^{\bullet\bullet\bullet i} D^{VW,ut} s_{Ut} + g_{Tuvw} D^{VWu,vtw} s_{Ut} - g_{i\bullet j}^{\bullet\bullet j} D^{VW,tu} s_{Uu} s_{Tt} \\ & + 2g_{ij}^{\bullet\bullet ij} D^{VW,tu} s_{Ut} s_{Tt} + g_{i\bullet vw}^{\bullet i} D^{VWv,tuw} s_{Uu} s_{Tt} + 2g_{iv\bullet w}^{\bullet\bullet i} D^{VWv,tuw} s_{Uu} s_{Tt} \\ & + 0.5g_{vwx} D^{VWvw,utxy} s_{Ut} s_{Tu}, \end{aligned} \quad (24)$$

$$\begin{aligned} \langle \Psi_T^{\bullet U} | H | \Psi_V^{\bullet W} \rangle = & h_{TV} D^{UW} s_{Tt} + 2h_i^{\bullet i} D^{UW} s_{TV} + h_{tu} D^{Ut,Wu} s_{TV} + h_{Vu} D^{Uu,tW} s_{Tt} \\ & + h_{Tu} D^{Ut,uW} s_{Vt} + 2h_i^{\bullet i} D^{Uu,tW} s_{Vu} s_{Tt} + 2h_{vw} D^{Utv,uWw} s_{Vt} s_{Tu} + g_{TVi}^{\bullet\bullet\bullet i} D^{UW} \\ & - 2g_{TiV}^{\bullet\bullet\bullet i} D^{UW} + 2g_{ij}^{\bullet\bullet ij} D^{UW} s_{TV} + g_{i\bullet tu}^{\bullet i} D^{Ut,Wu} s_{TV} - 2g_{it\bullet u}^{\bullet\bullet i} D^{Ut,Wu} s_{TV} \\ & + 2g_{ij}^{\bullet\bullet ij} D^{UW} s_{TV} + g_{i\bullet u}^{\bullet i} D^{Ut,Wu} s_{TV} + 2g_{it\bullet u}^{\bullet\bullet i} D^{Ut,uW} s_{TV} + 0.5g_{tuvw} D^{Utu,Wvw} s_{TV} \\ & - g_{Vi\bullet u}^{\bullet\bullet i} D^{Uu,tW} s_{Tt} + 2g_{Viu}^{\bullet\bullet\bullet i} D^{Uu,tW} s_{Tt} + g_{Vuvw} D^{Uuv,tWw} s_{Tt} - g_{Ti\bullet u}^{\bullet\bullet i} D^{Ut,uW} s_{Vt} \\ & + 2g_{Tiu}^{\bullet\bullet\bullet i} D^{Ut,uW} s_{Vt} + g_{Tuvw} D^{Utu,vWw} s_{Vt} - g_{i\bullet j}^{\bullet\bullet j} D^{Uu,tW} s_{Vu} s_{Tt} \\ & + 2g_{ij}^{\bullet\bullet ij} D^{Uu,tW} s_{Vu} s_{Tt} - g_{i\bullet vw}^{\bullet i} D^{Uuv,tWw} s_{Vv} s_{Tt} + g_{iv\bullet w}^{\bullet\bullet i} D^{Uuv,tWw} s_{Vu} s_{Tt} \\ & + 0.5g_{vwx} D^{Utvw,uWxy} s_{Vt} s_{Tu}. \end{aligned} \quad (25)$$

In Eqs. (22)–(25), the expressions are given in tensor form and the Einstein convention is used. **D** stands for RDM, **s**, **h**, and **g** are the overlap, and one- and two-electron integrals of VB orbitals. The summation indices are given in lowercase form, while the others are given in uppercase form.

C. The program procedure of the Newton-Raphson algorithm for VB orbital optimization

With the codes generated by the AFCG for the required Hamiltonian/overlap matrix elements, Eqs. (8)–(13), one can easily implement the Newton-Raphson algorithm into the

XMVB package,⁴³ which is an *ab initio* nonorthogonal VB program. The Newton-Raphson VBSCF optimization procedure includes the following steps:

1. Read in the initial guess orbitals.
2. Prepare the auxiliary orbitals by using the method developed in Paper II.²
3. Perform integral transformation from basis functions to nonorthogonal VB orbitals.
4. Compute auxiliary orbitals-based Hamiltonian and overlap matrices, and solve the secular equation in Eq. (3).
5. Compute RDMs up to the fourth-order with the methods described in Papers I and II.^{1,2}
6. Compute gradients and Hessian matrix by using the code generated by AFCG.
7. Solve the Newton-Raphson equation, Eq. (16), to obtain the updated VB orbitals, Eq. (15).
8. If the criterion of convergence is achieved, print out the output file; otherwise, go to Step 3 with the updated orbitals.

IV. TEST CALCULATIONS

The developed AFCG was validated to ensure that it is bug-free and the generated formulas and codes are correct. To this end, first, the formulas of energy gradients generated by AFCG are compared with the hand-derived ones presented in Paper II.² It is found that all formulas for energy gradients are identical. Furthermore, a test calculation of the N_2 molecule, using 6-31g* basis and 17 VB structures, and with HAOs, was performed. The values of gradients obtained by AFCG and by our previous code are exactly the same. Second, to check the correctness of the formulas and codes for Hessian, matrix elements of Eqs. (22)–(25) were calculated by the definition, namely that the icSEWF and icDEWF are expressed explicitly as the linear combinations of Slater determinants, and the matrix elements are evaluated through their corresponding matrix elements between these Slater determinants. Once again, the matrix elements computed by this straightforward way are identical to the AFCG implemented ones. Thus, we believe that AFCG is robust and reliable.

Two molecules, cyclobutadiene (C_4H_4) and benzene (C_6H_6), are selected for testing the AFCG-aided VBSCF rou-

tine, due to their important roles in organic chemistry. These two molecules are prototype molecules of the anti-aromatic and aromatic compounds.⁴⁸ The geometry optimization is carried out at the HF/cc-pVTZ level. For VBSCF, the cc-pVTZ basis set⁴⁹ is used. All the calculations were performed with the XMVB package⁴³ on a workstation with Intel Xeon E5-2690 v2 CPU and 320 GB memory.

Two types of orbitals, strictly localized HAOs and the semi-localized OEOs, are used in the VB calculations. For clarity, the corresponding VB calculations are labeled as VB/HAO and VB/OEO, respectively. In this paper, for the purpose of comparing our algorithms conveniently, a one-structure VBSCF calculation is performed first and the optimized orbitals are taken as the initial guess orbitals for the later calculations.

A. The comparison of the results of LBFGS and Newton-Raphson algorithm

Table I shows the performance comparison of the Newton-Raphson algorithm and the gradient-based LBFGS algorithm. As expected, for the VB/HAO calculations, it can be seen from Table I that the Newton-Raphson algorithm needs only 3-5 iterations to achieve convergence for all test molecules, while LBFGS needs much more iterations, varying from a few to more than one hundred. The differences in the numbers of iterations between these two algorithms become even larger when OEOs are used. For the CPU time, the Newton-Raphson algorithm requires much more computational time for each iteration due to the computation of the Hessian matrix. As a result, the total CPU time shows the mixture of these two factors. For those calculations in which the core orbitals are frozen, although the LBFGS algorithm requires more steps to converge, the difference of the number of iteration steps is not crucial, and then LBFGS algorithm requires less total CPU time for the whole calculation. But for those calculations without core-frozen approximation and using OEOs, due to the larger numbers of basis and orbitals, the LBFGS algorithm requires hundreds of iterations to get the final result, while the number of iterations for the Newton-Raphson algorithm does not change much, showing its advantage in the total CPU time. This indicates that the

TABLE I. The performance benchmark of the Newton-Raphson and the LBFGS algorithms.

Orbital type	Molecule	Newton-Raphson			LBFGS		
		Number of iteration	Total time (s)	Time per iteration (s)	Number of iterations	Total time (s)	Time per iteration (s)
HAO	$C_4H_4^a$	3	10.62	3.54	12	0.21	0.02
HAO	$C_6H_6^a$	5	14.11	2.82	10	1.02	0.10
OEO	$C_4H_4^a$	6	11.17	1.86	44	0.71	0.02
OEO	$C_6H_6^a$	6	34.89	5.82	59	4.67	0.08
HAO	C_4H_4	4	223.00	55.75	123	743.19	6.04
HAO	C_6H_6	5	1837.81	367.56	99	3213.85	32.46
OEO	C_4H_4	6	528.36	88.06	202	1316.27	6.52
OEO	C_6H_6	7	5026.90	718.13	235	8864.12	37.72

^aThe core orbitals are frozen during optimization.

TABLE II. The comparison of convergence behaviors between the Newton-Raphson and the LBFGS algorithms (C_6H_6 , with OEOs).

Newton-Raphson			LBFGS		
Iteration	Error ^a	$\ \xi\ $ ^b	Iteration ^c	Error ^a	$\ \xi\ $ ^b
1	0.151360	0.947338	1	0.151360	0.947338
2	0.015068	0.144551	3	0.015672	0.636531
3	0.003164	0.046934	17	0.003183	0.147568
4	0.000465	0.014368	56	0.000469	0.06724
5	0.000014	0.002910	191	0.000011	0.005614
6	0.000000	0.000108	219	0.000000	0.001914
7	0.000000	0.000009	235	0.000000	0.001120

^aError (in E_h) in the energy (compared to the converged one) of each iteration.^bThe Euclidean norm of the gradient.^cThe iteration with the minimum energy difference with respect to Newton-Raphson algorithm at the same row.

Newton-Raphson algorithm is able to keep satisfactory convergence behavior even when the number of optimized variables is large.

Table II shows the convergence behaviors of both Newton-Raphson and LBFGS algorithms. From this table, it can be seen that the Newton-Raphson algorithm converges more rapidly than the LBFGS algorithm, and the former has a better convergence quality. The LBFGS algorithm needs much more iterations than the Newton-Raphson algorithm to achieve the same energy. This is because the Newton-Raphson algorithm converges quadratically, while the LBFGS algorithm shows only linear or super-linear convergence behavior. The quadratic convergence behavior is particularly important for the VBSCF calculations in which thousands of variables are involved in optimization, such as VB/OEO calculation with large basis set. Meanwhile, it can be seen that the Newton-Raphson algorithm is able to converge quickly and to satisfy higher convergent criteria, and thus the result is more credible.

B. The features of Newton-Raphson algorithm

Fig. 1 illustrates how gradient components distribute and how this distribution varies during the Newton-Raphson optimization process for the VBSCF calculation of C_6H_6 with OEOs. At the beginning, the gradient components spread from about -0.2 to 0.3 . Then, as the function is being optimized, the distribution area becomes smaller and smaller. In the end, when the calculation converged, all the components of gradient are very close to zero (less than 10^{-7}), and all the point in the picture forms a line at coordinate zero of y axis. From Fig. 1, one can also see that in contrast to the well converged behavior of the Newton-Raphson algorithm, the gradient distribution of the gradient-based LBFGS algorithm still spread in a wider area from -0.0003 to 0.00009 , although the converged energies of these two algorithms are the same within six digits as can be seen from the last row in Table II. Thus the Hessian-based algorithm developed in this work not only converges more quickly but also converges more rigorously to the minimum than the gradient-based LBFGS algorithm does.

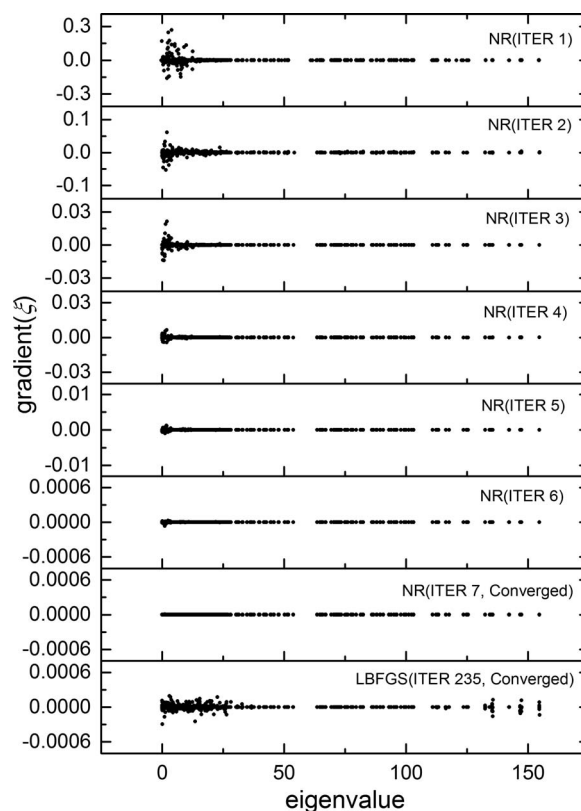


FIG. 1. The plots of gradient versus eigenvalue for the Newton-Raphson (each iteration) and LBFGS (converged) algorithms for the C_6H_6 calculation with 5 VB structures and OEOs.

Fig. 2 plots several quantities as a function of iteration in the Newton-Raphson optimization process for the VB calculation of C_6H_6 with 5 VB structures and OEOs: (a) the gradient norm, $\|\xi\|$, (b) the maximum component of the gradient $\xi_{\max}$, (c) the number of removed eigenvalues N_{rem} (with a threshold of 10^{-3}), and (d) the number of negative eigenvalues ($< -10^{-6}$). Figs. 2(a) and 2(b) show that both the norm and maximum component of the gradient drop down to zero as the iteration procedure processes. From Fig. 2(c), it can be seen that N_{rem} is bigger than the number of redundant variables at the first two iterations, as the wave function is far from the minimal point. In the rest five iterations, all removed eigenvalues come from the redundant variables. Fig. 2(d) shows that at the first four iterations, the energy Hessian is not semi-positive with dozens to hundreds of negative eigenvalues. This mainly comes from the coupling between the redundant and nonredundant variables. However, when the calculation converged, Fig. 2(d) shows clearly that there is no negative eigenvalue, leaving only zero eigenvalues coming from the redundant variables. These indicate that the calculation converges well to the minimal.

We now turn our attention to the ratio parameter of the predicted energy change by Eq. (21) and the actual one of the energy difference between two successive iterations. Table III collects the values of this quantity for the OEO calculation of benzene during the Newton-Raphson iterations. From Table III, it can be seen that the ratio parameters vary around 1.0 at the first few iterations and it becomes very close to 1.0 at convergence. The ratio parameter decreases

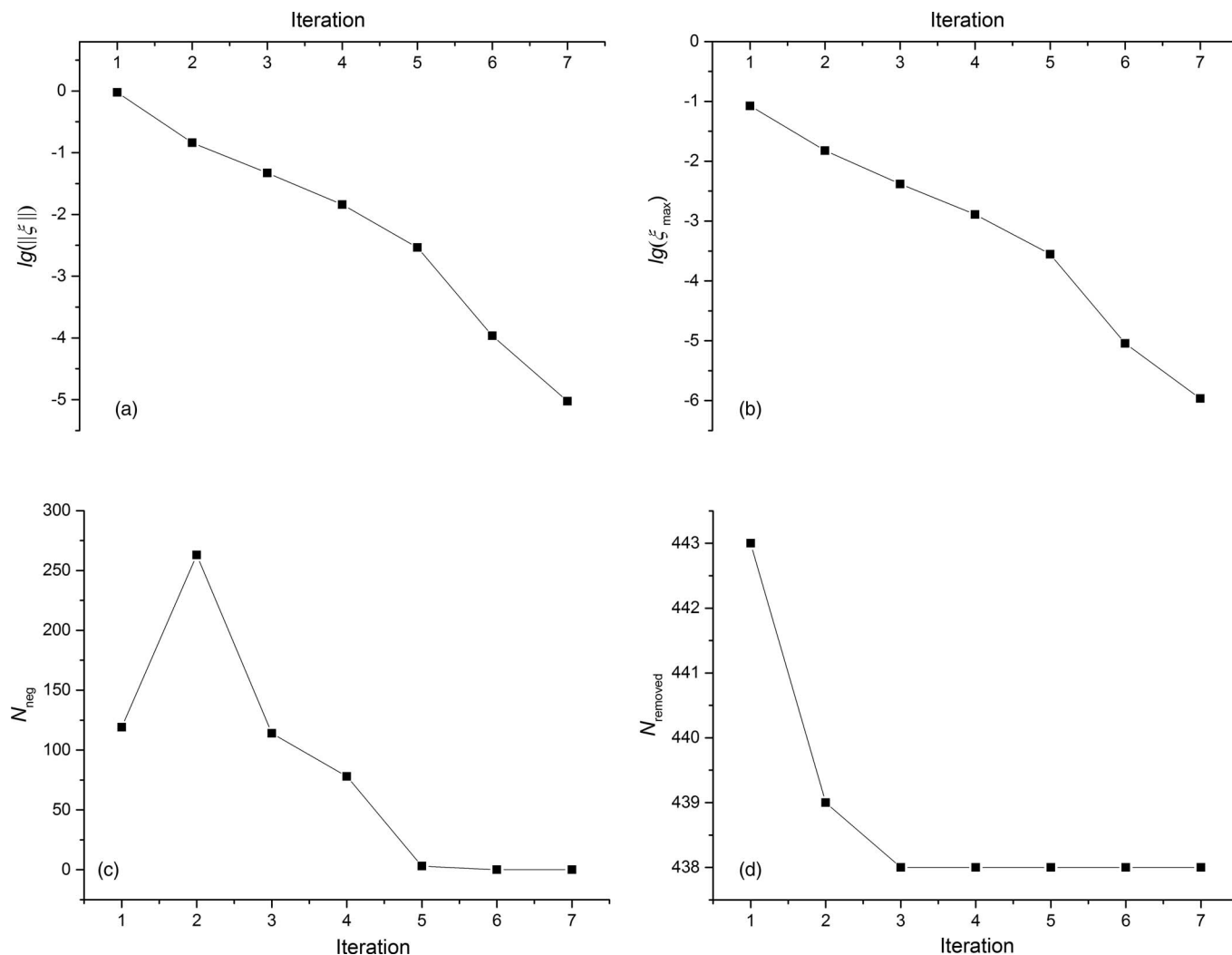


FIG. 2. The norm of the gradient $\|\xi\|$ (a), the maximum component of the gradient $\xi_{\max}$ (b), the number of removed eigenvalues N_{rem} (with a threshold of 10^{-3}) (c), and the number of negative (less than 10^{-5}) eigenvalues (d) in the Newton-Raphson optimization process of VBSCF calculation of C_6H_6 with 5 covalent VB structures and OEOs.

from 1.03 to 0.83 and then steadily increases to 1.0. These results show that the quadratic model is a good approximation near the stationary point, because the ratio parameter, which is close to 1.0, indicates the harmonicity of the energy surface. Despite that the energy function has some anharmonic character at the first few iterations, the algorithm presented in Sec. II, in which the negative and small eigenvalues are removed, still works correctly as the smallest ratio parameter (0.83) is not too far from 1.0.

C. Scaling and timing

Table IV shows the CPU time per iteration of various procedures in the Newton-Raphson algorithm. Four most time-consuming steps are listed in the table, which are electron integral transformation, matrix element calculation, gradient and Hessian matrix transformation, and linear equation solving, respectively.

From Table IV, it is clear that the CPU time for the evaluation of matrix elements in the VBSCF energy Hessian is

significantly smaller than the other parts for all the test calculations. It can also be seen from Table IV that the integral transformation and linear equation solving are the most time-consuming steps for the all calculations. This is because the scaling for integral transformation is $n_0 m^4$, while the scaling for the diagonalization of the Hessian matrix, which is performed for solving the linear equation, is about $(n_0 m)^3$,

TABLE III. The actual energy change and predicted energy change during the Newton-Raphson procedure for VBSCF calculation of C_6H_6 with OEOs. (Energies are given in E_h .)

Iteration	Actual	Predicted	Ratio
2	-0.136292	-0.139664	1.025
3	-0.011904	-0.009876	0.830
4	-0.002699	-0.002248	0.833
5	-0.000451	-0.000409	0.907
6	-0.000014	-0.000014	1.000
7	-0.000000	-0.000000	1.000

TABLE IV. The average CPU time (in second) per iteration of various procedures in the Newton-Raphson algorithm.

Orbital type	Molecule	Integral transformation	Matrix elements	Gradient and Hessian transformation	Linear equation solving
HAO	C ₄ H ₄ ^a	0.39	0.21	0.08	0.67
	C ₄ H ₄	20.17	1.62	5.01	29.61
	C ₆ H ₆ ^a	2.80	1.21	0.03	0.63
	C ₆ H ₆	103.45	9.67	31.15	230.29
OEO	C ₄ H ₄ ^a	0.45	0.24	0.30	0.79
	C ₄ H ₄	19.96	1.61	6.21	61.25
	C ₆ H ₆ ^a	2.93	0.67	0.09	0.87
	C ₆ H ₆	104.68	9.66	37.33	575.47

^aThe core orbitals are frozen during optimization.

where n_o and m are the numbers of occupied orbitals and basis functions, respectively. Clearly, to further improve the performance of the Newton-Raphson algorithm, a more efficient algorithm for solving the linear equation, Eq. (16), is required. Fortunately, this can be done by using iterative linear equations solver,⁵⁰ with which the diagonalization of Hessian matrix is avoided. The implementations of such improvements are currently under development.

V. COMBINE THE NEWTON-RAPHSON ALGORITHM AND THE LBFGS ALGORITHM

The LBFGS algorithm converges poor but consumes less CPU time for each iteration, while the Newton-Raphson algorithm requires more CPU time but converges quadratically near the minimum when the Hessian matrix is positive definite. Like strategy developed by Rashid and van Lenthe,³³ we could combine these two methods to promote their advantages. In this approach, the LBFGS algorithm is first applied and then it is switched to Newton-Raphson algorithm after some criteria are satisfied. In this work, the norm of the gradient is used as the criterion for the switching of these two algorithms and is set to the value of 0.1.

Table V shows the iteration process of this combined approach, together with the total CPU time. With this approach, the total CPU time for the calculation (3258.3 s) is less than

TABLE V. The performance of Newton-Raphson algorithm combined with LBFGS algorithm for the VBSCF calculation of C₆H₆ with OEOs.

Algorithm ^a	Iteration	Error ^b	$\ \xi\ $ ^c
LBFGS (674.5)	1	0.151362	0.947339
	5	0.013579	0.518979
	10	0.005981	0.111271
	18	0.002998	0.080025
Newton-Raphson (2583.8)	19	0.000200	0.011892
	20	0.000002	0.001395
	21	0.000000	0.000023

^aThe number in parentheses is the total CPU time (in second) for the algorithm.^bError (in E_h) in the energy (compared the converged one) of each iteration.^cThe Euclidean norm of the gradient.

the situations when either two optimization algorithms are applied solely (5026.9 and 8864.1 s).

VI. CONCLUSION

In this paper, the formulation of Hessian matrix with respect to VB orbitals was presented by using the RDM approach for VB theory. To accomplish the formulation, which is very tedious due to the use of nonorthogonal VB orbitals, an automatic formula/code generator (AFCG) is developed. With the aid of AFCG, the formulas for the Hamiltonian and overlap matrix elements between various internally contracted wave functions are generated automatically. The AFCG is able to optimize formulas so that all formulas are presented in a legible manner, outputted as elegant LaTeX files. Furthermore, the formulas are automatically implemented into Fortran codes, which were included in the XMVB package as a new module, the Hessian-based VBSCF module.

Test calculations showed that the Newton-Raphson algorithm shares quadratic convergence behavior and is very efficient for VBSCF calculations, particularly for the calculations that use OEOs and large basis sets. A combined approach with LBFGS and Newton-Raphson algorithms can be applied to further reduce the computational time. Moreover, the implementation of Hessian-based VBSCF method demonstrates again the efficiency of the nonorthogonal RDM-based approach, without which the evaluation of Hessian matrix elements would be very time consuming. Moreover, if the Newton-Raphson equation is solved by iterative method, the computational cost of current algorithm can be further reduced to only $n_o m^4$ for most VBSCF calculations with small active space, in which the most time consuming part is the integral transformation. It is worthwhile to note that the most time consuming step in the MO-based MCSCF method is also the integral transformation, provided that the active space is not too large. Thus in this case, the VBSCF method will share the same computational scaling as the MCSCF method.

In addition, test calculation shows that the CPU time for each iteration in the Newton-Raphson algorithm is still larger than in the LBFGS algorithm due to the higher scaling for the

electron integral transformation in the evaluation of Hessian. This can be improved by developing an optimization algorithm based on approximate Hessian. This improvement may be easily made with the help of AFCG and will be published in the near future.

It is also worthwhile to mention that apart from the implementation of the Hessian matrix, the AFCG is in fact a universal tool for nonorthogonal orbitals-based approaches, which is able to derive expressions and then to generate codes for matrix elements between internally contracted wave functions with any excitation orders. As a consequence, the AFCG could be treated as a generic tool not only for VB methods but also for other nonorthogonal orbital based methods. Furthermore, the AFCG would be used not only for the SCF methods, in which the gradients and Hessian are required, but also for the post-SCF methods, where internally contracted wave functions are involved in the many-electron wave functions, such as many-body methods based on perturbation theory and coupled-cluster theory.

ACKNOWLEDGMENTS

This project is supported by the Natural Science Foundation of China (Grant Nos. 21120102035, 21273176, and 21290190) and Ministry of Science and Technology of China (Grant No. 2011CB808504).

APPENDIX: THE IMPLEMENTATION OF AFCG

1. General features

The AFCG is implemented with the Python programming language.⁴⁶ As a scripting language, Python allows programmers to express concepts in fewer lines of code than in other languages, such as C and Fortran. Furthermore, the object-oriented feature and its large standard library make it a very suitable programming language for developing AFCG. The disadvantage of Python may be the performance, due to the interpretive execution property of the scripting language. But because the formulas and codes are generated only once, the performance of the generated codes should be the first priority to be concerned.

The AFCG is implemented as a command line program running under Linux, and the input and output are prepared and given in the plain text form. Taking the Hamiltonian matrix elements between two icSEWFs as an example, the input plain text form of AFCG is represented as

$$\langle \Psi_q^{\bullet p} | H | \Psi_r^{\bullet s} \rangle \rightarrow \text{Psi_q}^{\wedge p} | H | \text{Psi_r}^{\wedge s},$$

where “_” indicates subscript, while “^” indicates superscript.

Following the same representation rules, AFCG can generate the output formulas in the plain text form. However, for the convenience of human reading, the output formulas are further transferred into the LaTeX input file format, and then compiled into a PDF file. The output codes are in standard Fortran form and are compiled into the XMVB package⁴³ with a general Fortran compiler such as GFortran or ifort.

2. Formula generation

Once the text input file is ready, it is sent to the automatic formula generator (AFG) to generate the resulting formulas. By applying the enhanced Wick's theorem,¹ one is able to deduce the formulas of matrix elements conveniently. The AFG fulfills the formulation in the following three steps:

1. Apply the enhanced Wick's Theorem to the input matrix element to derive the preliminary result formulas.
2. Use the orthogonality properties of orbitals to further simplify the formulas.
3. Combine similar terms and delete vanishing terms to tidy up the formulas, and then output the final resulting formulas.

The resulting formulas might be very complicated, including numerous terms that involve many summations over orbital indices. But it is not difficult to find out that all terms share a common pattern, that is, a chain of tensor contractions between electronic integrals and RDMs. Thus, the most important factor in the implementation is to design an efficient algorithm for tensor contractions.

3. Code generation

A classical Fortran subroutine can usually be divided into three parts: the variable declaration, the variables initialization, and the execution part. The design of the automatic code generator (ACG) also follows this convention and constructs these three parts automatically. Take the overlap matrix of the VBSCF wave function and icDEWF, Eq. (A1), as an example, first, all the tensors, including one 2-RDM, one overlap matrix and one intermediate tensor, and together with their ranks and sizes, are declared. Second, all the declared variables are initialized with the corresponding values. After that, the tensor contractions are accomplished by successively contracting the first two tensors and putting the result into an intermediate tensor. The following equation describes this process:

$$\begin{aligned} \langle \Psi | \Psi_{TU}^{\bullet \bullet VW} \rangle &= D^{VW, ut} s_{U_t} s_{T_u} \\ &= (D^{VW, ut} s_{U_t}) s_{T_u} \\ &= M_U^{\bullet VWu} s_{T_u} \quad (M_U^{\bullet VWu} = D^{VW, ut} s_{U_t}) \\ &= R_{TU}^{\bullet \bullet VW}. \end{aligned} \quad (\text{A1})$$

Apparently, the computational cost depends on the order of contraction, which we shall discuss in Subsection A 4 of the Appendix. The tensor contraction between two tensors is finished by two subroutines, SORTTENSOR and CONTRACTTENSOR, which are prepared manually. Subroutine SORTTENSOR sorts the indices of the tensors. It moves the summation indices together to the end. Once this is done, the several summation indices can be represented with a single number by using the multilevel system and then the contraction of these two tensors can be accomplished by CONTRACTTENSOR, which is nothing else but a classical matrix multiplication routine.

The sample output code of Eq. (A1) is given as follow:

Codes1.F90

```
! P^VWut * s_Ut * s_Tu
subroutine Cal_TUVW (Ni, Na, Nv, Res)
  ! Variables declaration
  integer :: D2_rank
  .....
  ! Variables initialization
  D2_rank = 2
  .....
  ! sort and contract tensors
  Call SORTTENSOR(D2)
  Call SORTTENSOR(S_Ut)
  Call CONTRACTTENSOR(D2, S_Ut, D2_S_Ut)
  Call SORTTENSOR(D2_S_Ut)
  Call SORTTENSOR(S_Tu)
  Call CONTRACTTENSOR(D2_S_Ut, Res)
  .....
End subroutine Cal_term1
```

4. Code optimization

The implementation of AFCG is simple and straightforward. However, to improve the performance of the resulting code, the following three techniques are applied to optimize the automatic generated codes.

1. Tensor contraction order optimization

Because the size of the tensors and the number of the summation indices are different in different tensors, the order of contraction is a key factor for the computational cost. The AFCG is able to find the most optimized order to reduce the computational cost.

Taking the tensor contractions in Eq. (A1) as an example, instead of doing the contraction with

$$(D^{VW,ut} s_{U_t}) s_{T_u}, \quad (\text{A2})$$

one could also contract these tensors with a different order,

$$(s_{U_t} s_{T_u}) D^{VW,ut}. \quad (\text{A3})$$

It is not difficult to figure out that the contraction order in Eq. (A2) is more efficient.

2. Common sub-expression elimination

The resulting expressions for different matrix elements may contain the same sub-expressions. AFCG is able to scan all the formulas to find all the common expressions and then compute them once for all matrix elements that share the same sub-expressions. This technique is called common sub-expression elimination and is widely used in the field of compiler optimizations.⁵¹

3. Symmetry consideration

The symmetry properties of the matrix elements are fully considered, and the symmetry of the electron integrals and the density matrix is taken into account when doing tensor

contractions so that enormous redundant calculations could be eliminated.

- ¹Z. Chen, X. Chen, and W. Wu, *J. Chem. Phys.* **138**(16), 164119 (2013).
- ²Z. Chen, X. Chen, and W. Wu, *J. Chem. Phys.* **138**(16), 164120 (2013).
- ³Z. Chen, X. Chen, F. Ying, J. Gu, H. Zhang, and W. Wu, *J. Chem. Phys.* **141**(13), 134118 (2014).
- ⁴J. H. van Lenthe and G. G. Balint-Kurti, *Chem. Phys. Lett.* **76**, 138–142 (1980).
- ⁵L. Song, J. Song, Y. Mo, and W. Wu, *J. Comput. Chem.* **30**(3), 399–406 (2009).
- ⁶R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, *SIAM J. Sci. Comput.* **16**(5), 1190–1208 (1995).
- ⁷J. Hinze and C. C. J. Roothaan, *Prog. Theor. Phys. Suppl.* **40**, 37–51 (1967).
- ⁸G. Das, *J. Chem. Phys.* **58**(11), 5104–5110 (1973).
- ⁹J. Hinze, *J. Chem. Phys.* **59**(12), 6424–6432 (1973).
- ¹⁰B. O. Roos, in *Lecture Notes in Quantum Chemistry* (Springer, 1992), pp. 177–254.
- ¹¹E. Dalgaard and P. Jørgensen, *J. Chem. Phys.* **69**(8), 3833–3844 (1978).
- ¹²D. L. Yeager and P. Jørgensen, *J. Chem. Phys.* **71**(2), 755–760 (1979).
- ¹³P. Jørgensen, P. Swannstro, and D. L. Yeager, *J. Chem. Phys.* **78**(1), 347–356 (1983).
- ¹⁴H. J. Werner and W. Meyer, *J. Chem. Phys.* **73**(5), 2342–2356 (1980).
- ¹⁵H. J. Werner and W. Meyer, *J. Chem. Phys.* **74**(10), 5794–5801 (1981).
- ¹⁶M. R. Hoffmann, D. J. Fox, J. F. Gaw, Y. Osamura, Y. Yamaguchi, R. S. Grev, G. Fitzgerald, H. F. Schaefer III, P. J. Knowles, and N. C. Handy, *J. Chem. Phys.* **80**(6), 2660–2668 (1984).
- ¹⁷P. E. Siegbahn, J. Almlöf, A. Heiberg, and B. O. Roos, *J. Chem. Phys.* **74**(4), 2384–2396 (1981).
- ¹⁸H. J. A. Jensen and H. Ågren, *Chem. Phys.* **104**(2), 229–250 (1986).
- ¹⁹B. H. Lengsfeld III, *J. Chem. Phys.* **73**(1), 382–390 (1980).
- ²⁰D. R. Yarkony, *Chem. Phys. Lett.* **77**(3), 634–635 (1981).
- ²¹R. Shepard, I. Shavitt, and J. Simons, *J. Chem. Phys.* **76**(1), 543–557 (1982).
- ²²W. A. Goddard III, *Phys. Rev.* **157**(1), 73 (1967).
- ²³J. Gerratt and W. Lipscomb, *Proc. Natl. Acad. Sci. U.S.A.* **59**(2), 332 (1968).
- ²⁴R. C. Ladner and W. A. Goddard III, *J. Chem. Phys.* **51**(3), 1073–1087 (1969).
- ²⁵D. Cooper, J. Gerratt, and M. Raimondi, in *Advances in Chemical Physics: Ab initio Methods in Quantum Chemistry-II* (John Wiley & Sons, Inc., 1987), Vol. 69, pp. 319–397.
- ²⁶D. L. Cooper, J. Gerratt, and M. Raimondi, *Chem. Rev.* **91**(5), 929–964 (1991).
- ²⁷N. Pyper and J. Gerratt, *Proc. R. Soc. London, Ser. A* **355**(1682), 407–439 (1977).
- ²⁸J. Gerratt and M. Raimondi, *Proc. R. Soc. London, Ser. A* **371**, 525–552 (1980).
- ²⁹P. B. Karadakov, J. Gerratt, D. L. Cooper, and M. Raimondi, *J. Chem. Phys.* **97**, 7637 (1992).
- ³⁰T. Thorsteinsson and D. L. Cooper, *Theor. Chim. Acta* **94**(4), 233–245 (1996).
- ³¹T. Thorsteinsson and D. L. Cooper, *Int. J. Quantum Chem.* **70**(4–5), 637–650 (1998).
- ³²T. Thorsteinsson and D. L. Cooper, *Mol. Phys.* **93**(4), 663–674 (1998).
- ³³Z. Rashid and J. H. van Lenthe, *J. Chem. Phys.* **138**, 054105 (2013).
- ³⁴C. L. Janssen and H. F. Schaefer III, *Theor. Chim. Acta* **79**(1), 1–42 (1991).
- ³⁵X. Li and J. Paldus, *J. Chem. Phys.* **101**(10), 8812–8826 (1994).
- ³⁶M. Nooijen, *Int. J. Mol. Sci.* **3**(6), 656–675 (2002).
- ³⁷A. A. Auer, G. Baumgartner, D. E. Bernholdt, A. Bibireata, V. Choppella, D. Cociorva, X. Gao, R. Harrison, S. Krishnamoorthy, and S. Krishnan, *Mol. Phys.* **104**(2), 211–228 (2006).
- ³⁸S. Hirata, *J. Phys. Chem. A* **107**(46), 9887–9897 (2003).
- ³⁹S. Krishnan, S. Krishnamoorthy, G. Baumgartner, D. Cociorva, C. C. Lam, P. Sadayappan, J. Ramanujam, D. Bernholdt, and V. Choppella, in *High Performance Computing - HiPC 2003*, edited by T. Pinkston and V. Prasanna (Springer, Berlin/Heidelberg, 2003), Vol. 2913, pp. 406–417; A. Bibireata, S. Krishnan, G. Baumgartner, D. Cociorva, C. C. Lam, P. Sadayappan, J. Ramanujam, D. Bernholdt, and V. Choppella, *Memory-Constrained Data Locality Optimization for Tensor Contractions* (Springer, USA, 2004); A. Hartono, A. Sibiryakov, M. Nooijen, G. Baumgartner, D. Bernholdt, S. Hirata, C. C. Lam, R. Pitzer, J. Ramanujam, and P. Sadayappan, in *Computational Science—ICCS 2005*, edited by

- V. Sunderam, G. van Albada, P. A. Sloot, and J. Dongarra (Springer, Berlin/Heidelberg, 2005), Vol. 3514, pp. 155–164; S. Hirata, *Theor. Chem. Acc.* **116**(1–3), 2–17 (2006); S. Hirata, *J. Phys.: Conf. Ser.* **46**, 249–253 (2006).
- ⁴⁰M. Valiev, E. J. Bylaska, N. Govind, K. Kowalski, T. P. Straatsma, H. J. J. Van Dam, D. Wang, J. Nieplocha, E. Apra, T. L. Windus, and W. A. de Jong, *Comput. Phys. Commun.* **181**(9), 1477–1489 (2010).
- ⁴¹J. T. Fermann and E. F. Valeev, A library for the evaluation of molecular integrals of many-body operators over Gaussian functions, 2003, see <http://libint.valeev.net/>.
- ⁴²J. H. van Lenthe and G. G. Balint-Kurti, *J. Chem. Phys.* **78**(9), 5699–5713 (1983).
- ⁴³L. Song, Z. Chen, F. Ying, J. Song, X. Chen, P. Su, Y. Mo, Q. Zhang, and W. Wu, XMBVB 2.0: An ab initio Non-orthogonal Valence Bond Program, Xiamen 361005, China, 2012; L. Song, Y. Mo, Q. Zhang, and W. Wu, *J. Comput. Chem.* **26**(5), 514–521 (2005).
- ⁴⁴T. Helgaker, P. Jorgensen, and J. Olsen, *Molecular Electronic-Structure Theory* (Wiley, 2013).
- ⁴⁵Z. Chen and M. R. Hoffmann, *J. Chem. Phys.* **137**, 014108 (2012).
- ⁴⁶G. V. Rossum, *The Python Language Reference Manual* (Network Theory Ltd., 2003).
- ⁴⁷See supplementary material at <http://dx.doi.org/10.1063/1.4901729> for all the 64 non-zero formulas for the matrix elements.
- ⁴⁸Y. Mo, W. Wu, and Q. Zhang, *J. Phys. Chem.* **98**(40), 10048–10053 (1994).
- ⁴⁹T. H. Dunning, *J. Chem. Phys.* **90**(2), 1007–1023 (1989).
- ⁵⁰G. H. Golub and C. F. Van Loan, *Matrix Computations* (Johns Hopkins University Press, 2012); J.-F. Bonnans, J. C. Gilbert, C. Lemaréchal, and C. A. Sagastizábal, *Numerical Optimization: Theoretical and Practical Aspects* (Springer, 2006).
- ⁵¹J. Cocke, *ACM SIGPLAN Not.* **5**(7), 20–24 (1970).